

LUNCH & LEARN

Building Your First Agent



What is an Agent?

An AI agent is a system that perceives its environment, reasons about it, and takes actions to achieve a goal – autonomously, in a loop.

Perceive

Receives input (text, emails, data, tool results) and understands context.

Reason

Uses a model to plan – a language model or RL model.

Act

Calls tools, APIs, or other agents to actually do something.

Every Agent Has 3 Parts

1

Model

The AI brain — Gemini, GPT, Claude, etc.
Does the reasoning and decision-making.

2

Tools

Functions the agent can call — APIs, search,
Gmail, databases, code execution...

3

Instructions

The system prompt. Tells the model who it is,
what it should do, and how to behave.

```
from google.adk.agents import LlmAgent

agent = LlmAgent(
    # 1. Model
    model="gemini-2.5-flash",

    # 2. Instructions
    instruction="""
        You are a helpful email assistant.
        When asked to check emails, use get_unread_emails
        to fetch them. Summarize each one briefly: who it
        is from and what they want. If an email clearly
        needs a reply, use draft_reply to prepare one
        and tell the user what you drafted.
        Never send emails. Always draft for human review.
    """,

    # 3. Tools
    tools=[
        read_gmail_tool,
        send_reply_tool,
    ]
)
```

Agents vs. Workflows

Both are agentic systems -- but they differ in who controls the flow.

Workflows

LLMs and tools orchestrated through predefined code paths.

- You define the steps upfront in code
- Predictable, consistent, easier to debug
- Best for well-defined, repeatable tasks
- Ex: prompt chaining, routing, parallelization

vs

Agents

LLMs dynamically direct their own processes and tool usage.

- The model decides what to do next
- Flexible -- handles open-ended problems
- Higher cost and latency, but more capable
- Best when steps can't be predicted upfront

What is Google ADK?

ADK stands for Agent Development Kit. It is Google's free, open-source toolkit that gives you everything you need to build an AI agent: a way to connect a model, give it tools, and run it.

The brain

Think of ADK as the platform that lets you connect Gemini (Google's AI) to your tools and give it a job to do. Like hiring a smart assistant and handing them a phone and a to-do list.

The framework

An open-source Python library. You define an LlmAgent with a model, instructions, and tools. ADK handles the loop, tool routing, session state, and deployment.

The connector

ADK plugs into Gmail, Google Calendar, Google Drive, Slack, and hundreds of other services out of the box. You do not need to build integrations from scratch.

What We're Building



What the agent can do:

- Fetch and summarize your 10 most recent unread emails
- Apply labels to important emails automatically
- Save preferences and remember them across sessions
- Check existing Gmail labels and create new ones if needed
- Draft replies to emails that need a response
- Never send anything — always drafts for human review

Set Up Your Workspace

A Install VS Code (your code editor)

1. Go to code.visualstudio.com 2. Download for your OS (Mac) 3. Install it and open it.

B Open the terminal inside VS Code

In VS Code: click Terminal in the top menu, then New Terminal. A black panel will open at the bottom. That is where you type commands.

C Create your project folder

```
cd Desktop
mkdir email_agent
cd email_agent
```

Files you will have by the end

<code>email_agent/</code>	your project folder
- <code>__init__.py</code>	makes it a Python package
- <code>.env</code>	your Google API key (Step 2)
- <code>agent.py</code>	the agent code (Step 5)
- <code>tools.py</code>	Gmail functions (Step 4)
- <code>credentials.json</code>	from Google Cloud (Step 3)
- <code>memory.json</code>	agent preferences (auto-created)
- <code>token.json</code>	Gmail auth token (auto-created)

Create `.env` now: paste `GOOGLE_API_KEY=your_key` inside it.

Steps 1 & 2: Set Up Python + Get Your Key

1

Create a virtual environment

Keeps your project packages separate from the rest of your computer.

```
# Run this inside your email-agent folder
python -m venv venv

# Activate it:
# Mac / Linux:
source venv/bin/activate

# Windows:
venv\Scripts\activate

# You will see (venv) appear in your terminal. That means it worked.
```

2

Install the packages

```
pip install google-adk python-dotenv google-auth-oauthlib google-api-
python-client
```

Step 3: Get a Gemini API Key

- 1 Go to aistudio.google.com
- 2 Sign in with your Google account
- 3 Click Get API key in the top left
- 4 Click Create API key
- 5 Copy the key it gives you

Then open your .env file and paste:

```
GOOGLE_API_KEY=paste_key_here
```


Step 3: Enable Gmail Access

3 Set up Gmail API + OAuth in Google Cloud

- a Go to `console.cloud.google.com` → Create a new project (e.g. my-email-agent)
- b APIs & Services → Library → Search "Gmail API" → Enable it
- c APIs & Services → Credentials → Create OAuth Client ID (App Type: Desktop app | Name: My Email Agent)
- d Download the JSON file → rename it `credentials.json` → put it in your project folder

 OAuth handles user consent — your agent will open a browser the first time to ask for permission. After that it saves a token locally.

OPTIONAL

Fixing the Consent Screen Error

Only needed if step 3c blocked you. Skip this slide if it worked fine.

1

Go to OAuth Consent Screen

In Google Cloud Console: APIs & Services → OAuth Consent Screen. Click Get Started.

2

Fill in the required fields

App name: anything like Email Agent. User support email: your email. Audience: External. Contact email: your email again at the bottom. Click Save and Continue.

3

Go back and create credentials

Click Menu. Now go to API& Services → Credentials → Create Credentials → OAuth Client ID and it will work.

The Tools We Give Our Agent

Each tool is a Python function the agent can call whenever it decides it needs to.

`get_unread_emails()`

Fetches your 10 most recent unread emails and returns who sent them, the subject, and a short preview.

`create_label()`

Creates a brand new label in Gmail if none of the existing ones fit. The agent names it based on the email content.

`draft_reply()`

Takes a thread and a message, and saves a reply as a draft in your Gmail. Never sends anything automatically.

`apply_label()`

Sticks a label onto a specific email. Only used on emails the agent decides are important enough to organize.

`get_labels()`

Returns all the labels that already exist in your Gmail inbox so the agent knows what categories are available.

All 5 tools live in `tools.py`. The agent decides which ones to call and in what order.

Giving the Agent a Memory

By default the agent forgets everything between sessions. These two tools fix that.

`save_preference()`

When the agent learns something useful — like which senders are always urgent — it calls this to write that to a file so it remembers next time.

`load_preferences()`

Called automatically at the start of every session. Reads back everything the agent has learned so far about how you like your inbox managed.

What `memory.json` looks like:

```
{
  "urgent_senders": "yb@scsp.ai",
  "ignore_labels": "newsletters",
  "draft_style": "formal and brief"
}
```

No black box.

Memory is just a plain text JSON file that lives on your own computer. You can open it, edit it, or delete it at any time. The agent only knows what you let it save.

Step 4: Define Your Tools

4

Tools = what your agent can DO in the world

```
def get_unread_emails() -> list:
    """Fetches the 10 most recent unread emails.
    Returns a list of subject + sender."""
    service = build_gmail_service()
    results = service.users().messages().list(
        userId='me',
        labelIds=['UNREAD'],
        maxResults=10
    ).execute()
    return parse_messages(results)
```

get_unread_emails

```
def draft_reply(
    thread_id: str,
    reply_text: str
) -> str:
    """Creates a draft reply to an email
    thread. Returns the draft ID."""
    service = build_gmail_service()
    draft = service.users().drafts().create(
        userId='me',
        body=make_draft(thread_id, reply_text)
    ).execute()
    return draft['id']
```

draft_reply

Step 5: Create the Agent

5

Wire the model, tools, and instructions together

```
import os
from dotenv import load_dotenv
from google.adk.agents import LlmAgent
from tools import get_unread_emails, draft_reply, get_labels, create_label, apply_label

load_dotenv()

root_agent = LlmAgent(
    model="gemini-2.5-flash",

    name="email_assistant",

    instruction="""
    You are a sharp, efficient email assistant who values the user's time.
    You have access to their Gmail. Be proactive – surface what matters,
    ignore what doesn't, and always draft before sending anything.
    """,

    tools=[
        get_unread_emails,
        draft_reply,
        get_labels,
        create_label,
        apply_label,
    ],
)
```

----- Gemini 2.5 Flash — fast and smart for tool use

----- Any name — used in multi-agent setups

----- The system prompt — this is where you add guardrails

----- Your custom tools from the previous step

Step 6: Run Your Agent

6

Two ways to run — terminal or web UI

Option A — Web UI (easiest)

```
# Outside your project folder:  
cd ~/Desktop  
adk web  
  
# Opens http://localhost:8000  
# Chat with your agent in a browser!
```

Option B — Terminal (for automation)

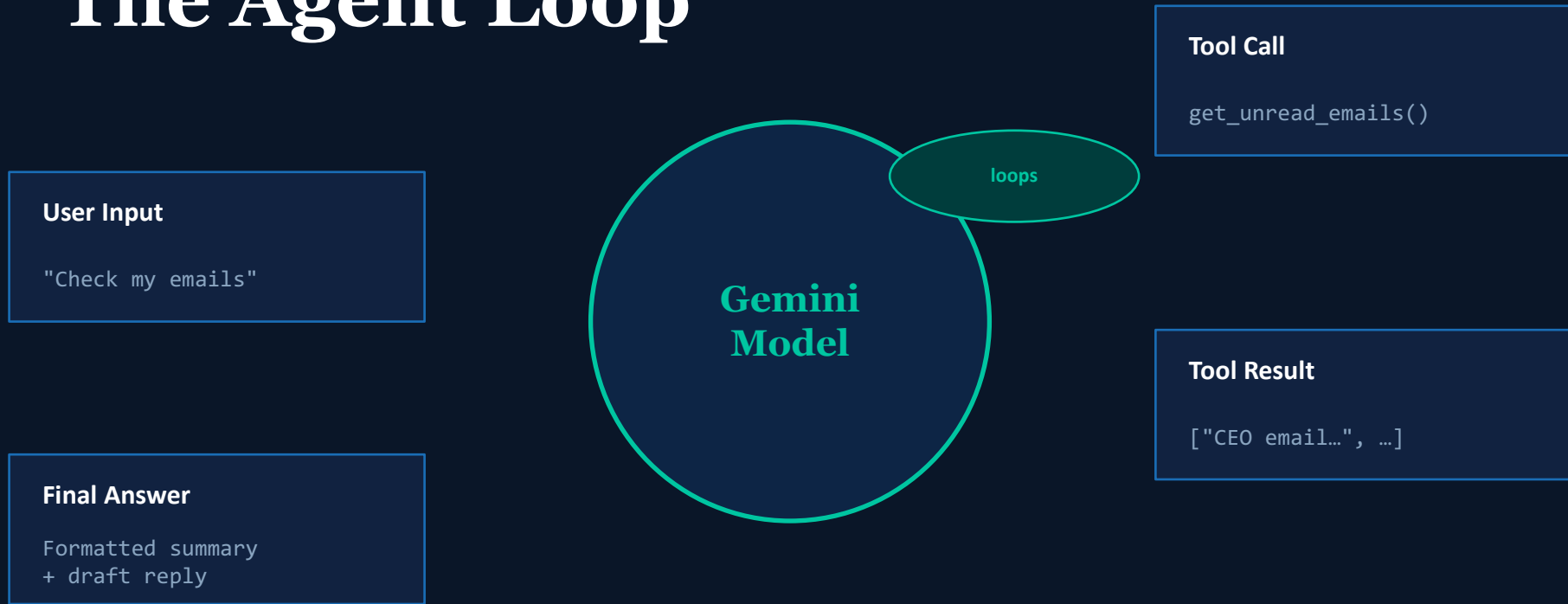
```
# Run interactively in the terminal:  
cd ~/Desktop  
adk run email_agent
```

What a conversation looks like:

You: "Check my emails and flag anything urgent."

Agent: "I found 4 unread emails. One from your CEO at 8 AM marked urgent — subject: 'Q3 Budget Review.' I've drafted a reply acknowledging receipt. 2 newsletters archived. 1 needs your input on a deadline — I've flagged it for you."

The Agent Loop



The model loops: Receive → Think → Act (tool call) → Observe result → Repeat until done → Respond

You built a real agent.

What we built

- tools.py with 7 tools: email, labels, memory
- agent.py wired to Gemini 2.5 Flash
- A memory system that persists across sessions

What is next

- Add a second agent for triage or drafting
- Connect Google Calendar as a new tool
- Deploy to Cloud Run

Resources

ADK Docs → adk.dev

ADK GitHub → github.com/google/adk-python

Gemini API Key → aistudio.google.com

Sample Agents → github.com/google/adk-samples

Questions?